

OpenTrack — Open-Source C# Bug Tracker

Research Report & Implementation Roadmap

Inspired by MantisBT | Built for Modern C# Cross-Platform Development

1. MantisBT Feature Inventory (What We Are Matching)

MantisBT is fully open source (GPL v2, GitHub: mantisbt/mantisbt), written in PHP.

Current stable version: 2.28.3 (May 2026).

Issue Management

- Report bugs with description, steps to reproduce, expected vs. actual behavior
- Unique ID per issue, persistent URLs
- Customizable issue pages (choose which fields appear)
- Multiple projects per instance
- Sub-projects and categories within projects
- Customizable issue workflow (status transitions)
- Full change/edit history audit trail
- Issue relationships (parent/child, duplicate, related, blocks/blocked-by)
- Relationship graphs (visual)
- Sticky issues (pin to top of list)
- Group actions (bulk edit/close/assign multiple issues)
- Public and private issues (visibility control)
- Public and private notes per issue

Issue Severity & Priority

- Configurable severity levels (e.g., trivial, minor, major, crash, block)
- Configurable priority levels (e.g., none, low, normal, high, urgent)
- Configurable reproducibility fields
- Configurable resolution types (fixed, won't fix, duplicate, etc.)

Search & Filtering

- Simple search (keyword)
- Advanced multi-field filter (status, severity, priority, assignee, reporter, date, category, project)
- Saved/shared filters (per-user and shared across team)
- Full-text search across all issue fields

User Management & Access Control

- Multi-level user roles: Viewer, Reporter, Updater, Developer, Manager, Administrator
- Different access level per user per project
- Public and private projects
- Anonymous (read-only) access option

- User self-registration with CAPTCHA and email confirmation
- Self-service password reset
- Authentication: local, LDAP, Active Directory, HTTP Basic

Notifications

- Email notifications on issue create/update/close
- Users can "monitor" (subscribe to) specific issues
- Configurable notification rules per event type
- In-app messaging between users about a specific issue

Attachments

- File attachments on issues and notes
- Stored on filesystem or in database (configurable)
- Attachment auto-preview for images

Reporting & Analytics

- Built-in reports: issues by status, by priority, by reporter, by assignee, by category
- Graphs/charts: resolution trends, open vs. closed
- Roadmap view (issues grouped by target version)
- Changelog view (resolved issues by version)
- Export to CSV, Excel, Word
- RSS feeds

Source Control Integration

- Link issues to commits (Git, SVN, Mercurial)
- Auto-resolve issues from commit messages (e.g., "Fixes #123")
- Changeset viewer

Custom Fields

- Add custom data fields to any project
- Types: text, numeric, date, dropdown, checkbox, radio button, email, URL
- Required/optional, visibility rules per field

Time Tracking

- Log time spent per issue
- View total time per issue and project summary

Versioning

- Define product versions (e.g., 1.0, 1.1, 2.0)
- Tag issues as "affects version" and "fix version"
- Roadmap groups issues by target fix version

Wiki & External Integration

- Optional wiki integration (DokuWiki, MediaWiki)

- Plugin architecture for extending any functionality

REST API

- Full REST API with API token authentication
- Base URL: /api/rest/
- CRUD for issues, projects, users, versions, filters
- Webhooks for external integration

Internationalization

- 49+ language localizations in MantisBT

Technical Stack (MantisBT)

- Language: PHP (procedural + partial OOP)
- UI: HTML/CSS/JavaScript + jQuery
- Database: MySQL, MariaDB, PostgreSQL, MS SQL, Oracle
- Web server: Apache/Nginx + PHP runtime
- Visualization: GraphViz, JpGraph

2. What We Will Build: OpenTrack

OpenTrack — a feature-complete, open-source bug tracker written in C# (.NET 9), deployable as a cross-platform web app and optionally as a desktop app.

Name Rationale

"OpenTrack" — open source, tracks bugs. Simple and professional.

License

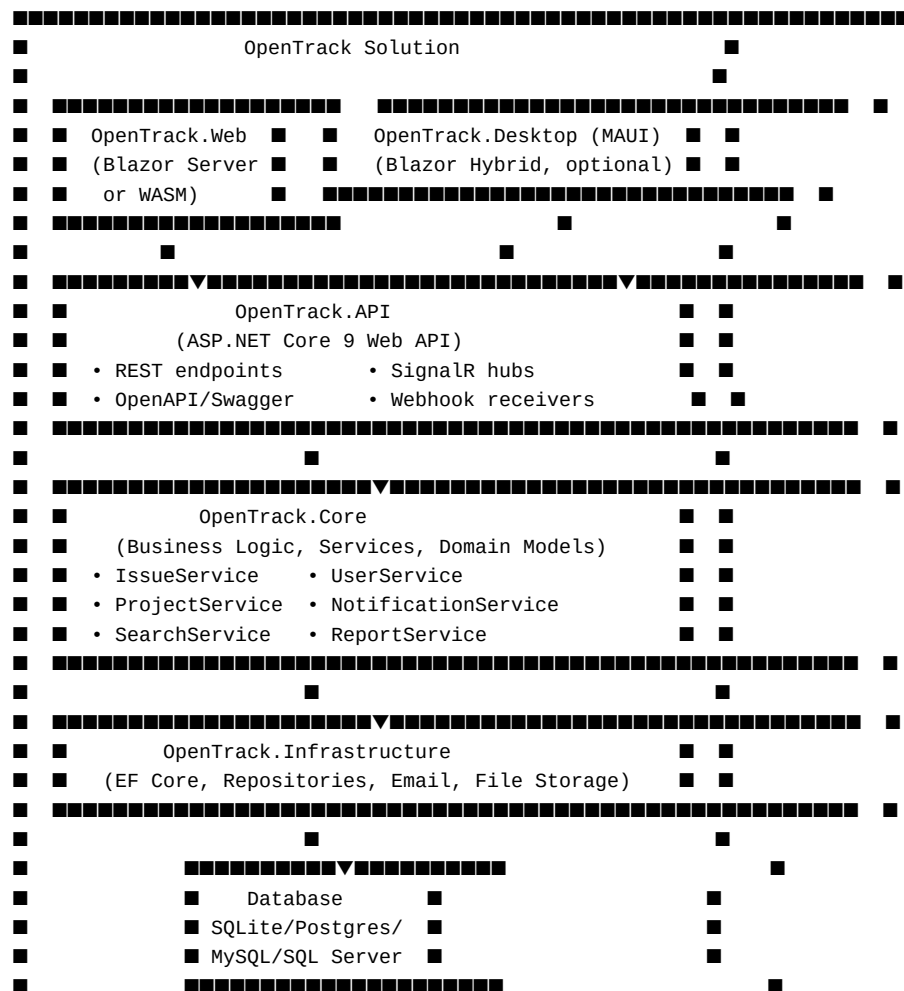
MIT License (more permissive than MantisBT's GPL v2, friendlier for embedding in other projects)

3. Technology Stack

Layer	Technology	Why
Language	C# 13 / .NET 9	Cross-platform, modern, type-safe
Web UI	Blazor Server (or Blazor WebAssembly)	Full C# front-end, no JS framework needed
Desktop (optional)	.NET MAUI Blazor Hybrid	Same Blazor components run natively
Backend API	ASP.NET Core 9 Web API	REST API, OpenAPI/Swagger auto-docs
ORM	Entity Framework Core 9	Code-first migrations, multi-DB support
Database	SQLite (default), PostgreSQL, SQL Server, MySQL	Flexible deployment

Layer	Technology	Why
Auth	ASP.NET Core Identity + JWT	Roles, claims, token auth
LDAP/AD	Novell LDAP / DirectoryServices	Enterprise auth
Email	MailKit	Cross-platform SMTP
File Storage	Local filesystem / Azure Blob / S3	Configurable
Search	EF Full-Text or Lucene.NET	Fast issue search
Charts	ApexCharts.Blazor or ChartJS via interop	Dashboard reporting
Realtime	SignalR	Live issue updates
Background Jobs	Hangfire	Email queues, scheduled reports
Source Control Webhooks	Custom middleware	GitHub, GitLab, Gitea hooks
Containerization	Docker + docker-compose	One-command deployment
CI/CD	GitHub Actions	Build, test, publish

4. Architecture Overview





Project Structure

```
OpenTrack/
├── src/
│   ├── OpenTrack.Core/           # Domain models, interfaces, business logic
│   ├── OpenTrack.Infrastructure/ # EF Core, repositories, email, storage
│   ├── OpenTrack.API/           # ASP.NET Core REST API + SignalR
│   ├── OpenTrack.Web/           # Blazor Server web front-end
│   └── OpenTrack.Desktop/       # .NET MAUI Blazor Hybrid (optional)
├── tests/
│   ├── OpenTrack.Core.Tests/
│   ├── OpenTrack.API.Tests/
│   └── OpenTrack.Web.Tests/
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── docs/
└── OpenTrack.sln
```

5. Feature-by-Feature Implementation Plan

Phase 1 — Foundation (Weeks 1–4)

Core infrastructure, auth, projects, and basic issue CRUD.

Database Models:

- User (Id, Username, Email, PasswordHash, Role, IsActive, Avatar)
- Project (Id, Name, Description, IsPublic, CreatedAt, OwnerId)
- ProjectMembership (UserId, ProjectId, Role)
- Category (Id, Name, ProjectId)
- Version (Id, Name, ProjectId, ReleaseDate, IsReleased, Description)
- Issue (Id, Title, Description, Steps, Expected, Actual, Status, Severity,

Priority, Reproducibility, Resolution, ProjectId, CategoryId, ReporterId,

AssigneeId, AffectsVersionId, FixVersionId, CreatedAt, UpdatedAt)

- IssueNote (Id, IssueId, AuthorId, Text, IsPrivate, CreatedAt)
- IssueHistory (Id, IssueId, UserId, FieldChanged, OldValue, NewValue, ChangedAt)
- IssueAttachment (Id, IssueId, FileName, FilePath, FileSize, ContentType, UploadedBy, UploadedAt)

Features Delivered:

- User registration, login (JWT + cookie)
- Create/edit/delete projects
- Create/edit/close/reopen issues
- Add notes to issues
- Basic issue list with status filter
- File attachments
- Role-based access (Viewer, Reporter, Developer, Manager, Admin)
- SQLite default database

Phase 2 — Power Features (Weeks 5–8)

Search, relationships, custom fields, notifications.

Additional Models:

- IssueRelationship (IssueId, RelatedIssueId, RelationType)
- CustomField (Id, Name, Type, ProjectId, IsRequired, Options)
- CustomFieldValue (IssueId, CustomFieldId, Value)
- IssueSponsor (IssueId, UserId, Amount, IsConfirmed)
- IssueMonitor (IssueId, UserId)
- NotificationRule (UserId, EventType, IsEnabled)
- TimeEntry (Id, IssueId, UserId, Hours, Description, LoggedAt)
- SavedFilter (Id, UserId, Name, IsShared, FilterJson)

Features Delivered:

- Advanced issue filter (all fields, save/share filters)
- Full-text search
- Issue relationships (parent/child/duplicate/blocks)
- Custom fields (text, numeric, date, dropdown, checkbox)
- Email notifications (MailKit + Hangfire queue)
- Issue monitoring (subscribe to changes)
- Time tracking per issue
- Bulk actions on multiple issues

Phase 3 — Reporting & Integrations (Weeks 9–12)

Dashboards, source control, REST API polish.

Features Delivered:

- Dashboard with charts (open/closed trends, by severity, by assignee)
- Roadmap view (issues by target version)
- Changelog view (resolved issues by version)
- CSV and Excel export
- RSS/Atom feed for issue updates
- GitHub/GitLab/Gitea webhook receiver (auto-link commits, auto-close on push)
- Full REST API (OpenAPI documented)
- API token management per user
- Webhook outgoing events (issue created/updated/closed → external URL)

Phase 4 — Enterprise Features (Weeks 13–16)

LDAP, wiki, plugin hooks, Docker publish.

Features Delivered:

- LDAP / Active Directory authentication
- Configurable workflow (custom status transitions per project)
- Plugin/extension hook system
- Multi-language / localization framework (English first, Spanish, German)
- Docker image + docker-compose for one-command deploy

- Desktop app (optional .NET MAUI Blazor Hybrid build)
 - Admin panel: instance settings, user management, email config
-

6. Key Domain Model (C# Sketch)

```
// Issue.cs
public class Issue
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public string Description { get; set; } = "";
    public string? StepsToReproduce { get; set; }
    public string? ExpectedBehavior { get; set; }
    public string? ActualBehavior { get; set; }

    public IssueStatus Status { get; set; } = IssueStatus.New;
    public IssueSeverity Severity { get; set; } = IssueSeverity.Minor;
    public IssuePriority Priority { get; set; } = IssuePriority.Normal;
    public IssueReproducibility Reproducibility { get; set; }
    public IssueResolution Resolution { get; set; } = IssueResolution.Open;

    public bool IsPrivate { get; set; }
    public bool IsSticky { get; set; }

    public int ProjectId { get; set; }
    public Project Project { get; set; } = null!;
    public int? CategoryId { get; set; }
    public Category? Category { get; set; }
    public int ReporterId { get; set; }
    public User Reporter { get; set; } = null!;
    public int? AssigneeId { get; set; }
    public User? Assignee { get; set; }
    public int? AffectsVersionId { get; set; }
    public int? FixVersionId { get; set; }

    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
    public DateTime? DueDate { get; set; }

    public ICollection<IssueNote> Notes { get; set; } = [];
    public ICollection<IssueAttachment> Attachments { get; set; } = [];
    public ICollection<IssueHistory> History { get; set; } = [];
    public ICollection<IssueRelationship> Relationships { get; set; } = [];
    public ICollection<CustomFieldValue> CustomFieldValues { get; set; } = [];
    public ICollection<TimeEntry> TimeEntries { get; set; } = [];
    public ICollection<IssueMonitor> Monitors { get; set; } = [];
}

public enum IssueStatus
{
    New = 10,
    Feedback = 20,
    Acknowledged = 30,
    Confirmed = 40,
    Assigned = 50,
    Resolved = 80,
```

```
Closed = 90
}

public enum IssueSeverity
{
    Feature = 10,
    Trivial = 20,
    Text = 30,
    Tweak = 40,
    Minor = 50,
    Major = 60,
    Crash = 70,
    Block = 80
}

public enum IssuePriority { None, Low, Normal, High, Urgent, Immediate }
public enum IssueReproducibility { Always, Sometimes, Random, HaveNotTried, UnableToReproduce, NA }
public enum IssueResolution { Open, Fixed, Reopened, UnableToReproduce, NotFixable, Duplicate, NotABug, Suspended, WontFix }
```

7. Advantages Over MantisBT

Feature	MantisBT (PHP)	OpenTrack (C#)
Language	PHP (legacy procedural)	C# 13 / .NET 9 (modern)
Cross-platform	Web-only (PHP + Apache)	Web + Desktop (Blazor + MAUI)
Real-time updates	No	Yes (SignalR)
Type safety	Partial (PHP)	Full (C# strong typing)
ORM / migrations	ADOdb (legacy)	EF Core (modern code-first)
Auth providers	LDAP, HTTP Basic	Identity + JWT + LDAP + OAuth2
REST API	Limited	Full OpenAPI 3.0 documented
Docker support	Third-party images	Official first-party
Desktop app	No	Yes (.NET MAUI optional)
License	GPL v2	MIT (more permissive)
Development language	PHP	C# (same as your project!)

8. Deployment Options

Self-hosted Web (primary):

```
docker-compose up -d
# Opens on http://localhost:5000
```

Windows standalone:

Single-file .exe with embedded Kestrel web server and SQLite.

No installation required — just run and open browser.

Linux / Mac:

Same dotnet publish output, runs on any platform .NET 9 supports.

Desktop (optional Phase 4):

.NET MAUI Blazor Hybrid — same Blazor pages running as a native Windows/Mac/Linux app.

9. Suggested GitHub Repository Structure

Repository: `github.com/YourName/OpenTrack`

License: MIT

Topics: `bug-tracker`, `issue-tracker`, `csharp`, `dotnet`, `blazor`, `open-source`

README will include:

- One-command Docker deploy
 - Feature screenshots
 - API docs link (Swagger UI)
 - Contributing guide
 - Comparison table with MantisBT
-

10. Recommended Build Order (Summary)

1. **Week 1–2:** Solution scaffold, EF Core models, Identity auth, SQLite, basic project/issue CRUD
 2. **Week 3–4:** Blazor UI — issue list, issue detail, create/edit forms, file upload
 3. **Week 5–6:** Search & filters, issue relationships, notes, history, monitoring
 4. **Week 7–8:** Custom fields, time tracking, email notifications, bulk actions
 5. **Week 9–10:** Dashboard charts, roadmap, changelog, CSV/Excel export
 6. **Week 11–12:** REST API polish, API tokens, GitHub/GitLab webhooks
 7. **Week 13–14:** LDAP auth, configurable workflow, admin panel
 8. **Week 15–16:** Docker packaging, optional MAUI desktop, docs, localization
-

Report generated: July 21, 2026

Research sources: MantisBT official docs, Wikipedia, MantisBT Wiki, Microsoft .NET docs